CubeTacToe

Prototyping to Enhance Employability

Godot 4 (Medieval) · Unreal Engine (Neon)

Medieval

Neon

Game Design · Engine Comparison · Portfolio Strategy

What Is CubeTacToe?

3D Tic-Tac-Toe on a cube - same universal rules, new strategic depth.

- Six faces, each a 3×3 grid - six simultaneous boards
- Commitment mechanic: first move locks the face for your second move
- Track threats across multiple faces at once
- Win conditions: Any Side or Majority (4 of 6 faces)
- Modes: Local 2P · vs AI (Easy/Medium/Hard) · Online (for Godot, Unreal is up to you ☺)

Why it works as a portfolio piece

- Rules explained in 30 seconds - reviewers focus on how, not what
- Same design, two engines - proves concept portability
- Every feature is a conversation starter:
    - Minimax AI with alpha-beta pruning
    - Peer-to-peer networking with RPCs
    - Background threading for responsive UI
    - HUD
- Finished & playable - shows shipping discipline

Rules & Game Loop

Choose Face → Place 1st Piece (face locks) → Place 2nd Piece (different face) → Check Win → Switch Player

Any Side

First to win any single face wins the game immediately.

Fast, aggressive games.

Majority

First to win 4 of 6 faces wins the game.

Longer, strategic battles.

Commitment Mechanic - Once you place your first piece, your second must go on a different face. This turns every opening move into a commitment - there are no safe neutral plays.

Design Document (Short Form)

Vision

A competitive strategy game that respects the player's intelligence - familiar enough to pick up in 30 seconds, deep enough to reward mastery.

Pillars

Spatial Thinking
Track 6 boards simultaneously; anticipate cascading wins.

Commitment
Locked-face forces planned aggression - no safe moves.

Tone Contrast
Same rules, two emotional registers: tavern bet vs neon duel.

Scope & Features

- ✓ Core rules
- ✓ Local 2-Player
- ✓ AI - 3 difficulty levels
- ✓ Online multiplayer (mostly)
- ✓ HUD
- ✓ Audio (music + SFX)
- ✓ Pause menu

UX Priorities

Camera
Cube auto-rotates to face the active board area.

HUD
Live face-win tracker, animated panel scaling, turn timer.

Feedback
Effects on placement, audio cues on win/loss.

Win Screen
Clear winner feedback

Two Versions - Same Rules, Different Worlds

Medieval

Godot 4 · C#

Tavern setting | Warm torchlight | Wooden UI frames

Key Technical Features

- C# scripting with Godot node system
- Minimax AI (alpha-beta, depth 3/6)
- Peer-to-peer online
- Tween-driven animated HUD
- Particle FX

Neon

Unreal Engine · Blueprint / C++

Cyberpunk setting | Emissives materials | Dark aesthetic

Key Technical Features

- Blueprint prototyping + C++ core systems
- Real-time neon glow
- No multiplayer (again, that's for you to do ☺)
- UMG-driven UI
- SM size changes on placement

Godot 4 vs Unreal Engine - Key Differences

Area	Godot 4	Unreal Engine
Editor size	~100 MB - instant setup	15–40 GB - heavy install
Scene model	Node tree - everything is a node	Actor / Component hierarchy
Scripting	GDScript or C#	Blueprints (visual) or C++
Iteration speed	Very fast - hot reload	Slower - shader compilation
Source control	Text .tscn files - git-friendly	Binary .uasset - hard to diff
Rendering	Good; manual shader/post setup	Lumen, Nanite - AAA out of box
Networking	Simple RPC via [Rpc] attribute	Server-authoritative, robust
Web export	✓ GDScript / ✗ C# not supported	Not supported
Console support	Community ports	Official first-party support

Verdict: Godot is faster to iterate in for prototypes. Unreal provides more for console/AAA targets. Knowing both is the point.

AI System - Minimax with Alpha-Beta Pruning

Difficulty Levels

Easy

Random move selection

Medium

Minimax · depth 3 · ~12 top moves per node

Hard

Minimax · depth 4-6 · background thread

Threading Strategy (Godot): Snapshot all Godot objects on the main thread → hand plain C# structs to Task.Run → await the result. The game loop keeps rendering and animating throughout the search.

Why This Strengthens an Employability Portfolio

01 Multi-Engine Fluency

Studios have preferred engines - but that changes between studios and sometimes mid-project. Demonstrating competence in Godot, Unreal or OTHER engines signals adaptability and engine-agnostic thinking.

02 Prototyping as a Skill

Prototyping quickly and intelligently is one of the most valued skills in games. A small, finished prototype shows scope control, systems thinking, and shipping discipline.

03 Same Design, Two Implementations

Rebuilding CubeTacToe in a second engine proves concept portability - understanding what the game is, independently from what the engine is. That's a senior-level mindset.

04 Concrete Technical Conversation Starters

Every feature is defensible in an interview: minimax algorithm, peer-to-peer networking, background threading, tween animation systems. Each one is evidence, not just a claim.

05 Breadth Without Sacrificing Depth

Small prototypes let you explore multiple engines, genres, and mechanics without spending 18 months on one abandoned project. Four polished small games beat one unfinished large game every time.

The Takeaway

CubeTacToe is a humble concept on purpose. The rules take 30 seconds to explain, so every minute of a portfolio review is spent on how you built it - not what it is.
Two engines. Two aesthetics. One design. That's the argument.

Repo: https://github.com/SMaimaris/CubeTacToeDuo.git